

Đăng ký C++ Types vào QML — Tổng hợp từ Qt 6 Docs

Điều kiện tiên quyết

Include header

```
cpp

// Nếu chỉ link QtQmlIntegration:
#include <QtQmlIntegration/qqmlintegration.h>

// Nếu đã link QtQml (phổ biến hơn):
#include <QtQml/qqmlregistration.h>
```

Khai báo trong build system

Tất cả các macro bên dưới chỉ có tác dụng khi file `.h` chứa class được đưa vào `qt_add_qml_module()` (CMake) hoặc được đánh dấu đúng cách (qmake).

CMake:

```
cmake

qt_add_qml_module(appcalculator
    URI calculator          # tên module dùng để import trong QML
    VERSION 1.0
    QML_FILES
        qml/Main.qml
    SOURCES
        src/calculatorengine.h src/calculatorengine.cpp
        #   ↑ file .h PHẢI nằm trong SOURCES – macro được đọc lúc compile
)
```

qmake:

```
pro

CONFIG += qmltypes
QML_IMPORT_NAME = calculator
QML_IMPORT_MAJOR_VERSION = 1

HEADERS += src/calculatorengine.h
SOURCES += src/calculatorengine.cpp
```

Lưu ý chung: header chứa class PHẢI accessible từ include path của project. Nếu không, code generation lúc build sẽ fail.

1. `QML_ELEMENT` — Instantiable type, tên = tên class

QML có thể tạo instance trực tiếp. Tên trong QML lấy **nguyên tên C++ class**.

cpp

```
// message.h
#include <QObject>
#include <QtQml/qqmlregistration.h>

class Message : public QObject {
    Q_OBJECT
    QML_ELEMENT // tên QML = "Message"
    Q_PROPERTY(QString author READ author WRITE setAuthor NOTIFY authorChanged)
public:
    explicit Message(QObject *parent = nullptr);
    QString author() const;
    void setAuthor(const QString &author);
signals:
    void authorChanged();
private:
    QString m_author;
};
```

CMake:

cmake

```
qt_add_qml_module(myapp
    URI com.mycompany.messaging
    VERSION 1.0
    SOURCES message.h message.cpp
)
```

Dùng trong QML — phải import:

qml

```
import com.mycompany.messaging // URI khai báo trong cmake

Message {
    author: "Amelie"
}
```

2. `QML_NAMED_ELEMENT(name)` — Instantiable type, đổi tên

Dùng khi tên C++ class có prefix (kiểu `CMyClass`) hoặc muốn tên QML khác tên C++.

```
cpp

// calculatorengine.h
class CCalculatorEngine : public QObject {
    Q_OBJECT
    QML_NAMED_ELEMENT(CalculatorEngine) // tên QML = "CalculatorEngine"
    Q_PROPERTY(QString result READ result NOTIFY resultChanged)
public:
    // ...
};
```

Dùng trong QML — phải import:

```
qml

import calculator

CalculatorEngine { // không phải CCalculatorEngine
    // ...
}
```

3. `QML_ELEMENT` + `QML_SINGLETON` — Singleton do engine quản lý

Engine tạo **đúng một instance**, owns và destroy nó khi engine bị destroy. Không cần làm gì trong `main.cpp`. Không có `QQmlContext` riêng.

```
cpp
```

```
// calculatorengine.h
class CalculatorEngine : public QObject {
    Q_OBJECT
    QML_ELEMENT
    QML_SINGLETON // engine tự tạo 1 instance duy nhất
    Q_PROPERTY(QString expression READ expression NOTIFY expressionChanged)
    Q_PROPERTY(QString result READ result NOTIFY resultChanged)
public:
    explicit CalculatorEngine(QObject *parent = nullptr);

    QString expression() const;
    QString result() const;

    Q_INVOKABLE void input(const QString &key);
    Q_INVOKABLE void clear();

signals:
    void expressionChanged();
    void resultChanged();

private:
    QString m_expression;
    QString m_result;
};
```

CMake — không khác gì type thường:

```
cmake

qt_add_qml_module(appcalculator
    URI calculator
    VERSION 1.0
    SOURCES
        src/calculatorengine.h src/calculatorengine.cpp
    QML_FILES
        qml/Main.qml
)
```

main.cpp — không cần làm gì thêm:

```
cpp
```

```
int main(int argc, char *argv[]) {
    QGuiApplication app(argc, argv);
    QmlApplicationEngine engine;
    engine.loadFromModule("calculator", "Main");
    return app.exec();
}
```

Dùng trong QML — import + gọi thẳng tên class, không tạo instance:

```
qml

import calculator                // import module

Text {
    text: CalculatorEngine.expression // bind thẳng vào singleton
}

Button {
    onClicked: CalculatorEngine.input("5") // gọi Q_INVOKABLE
}
```

Khác với QML singleton (`pragma Singleton`): `QML_SINGLETON` là C++ singleton do engine quản lý. `pragma Singleton` là QML singleton — file `.qml` chứa `pragma Singleton` và phải được khai báo là singleton trong CMake:

```
cmake

set_source_files_properties(qml/Config.qml
    PROPERTIES QT_QML_SINGLETON_TYPE TRUE)
```

4. `QML_ELEMENT` + `QML_UNCREATABLE(reason)` — Visible nhưng không tạo được

Type xuất hiện trong QML type system (có thể dùng làm kiểu property, dùng enum), nhưng QML **không được** tạo instance. Nếu cố tạo → runtime error với `reason`.

```
cpp
```

```
// filtertype.h
class FilterType : public QObject {
    Q_OBJECT
    QML_ELEMENT
    QML_UNCREATABLE("FilterType chỉ dùng để export enum, không tạo instance")
public:
    enum Type { All, Active, Completed };
    Q_ENUM(Type)
};
```

CMake:

```
cmake

qt_add_qml_module(myapp
    URI myapp
    VERSION 1.0
    SOURCES filtertype.h
    # không cần .cpp nếu class không có implementation
)
```

Dùng trong QML — phải import, dùng được enum:

```
qml

import myapp

// Dùng enum OK:
dispatcher.setFilter(FilterType.Active)

// KHÔNG được tạo instance:
// FilterType { } ← runtime error: "FilterType chỉ dùng để export enum..."
```

5. `QML_ANONYMOUS` — Không tạo được, không tham chiếu được từ QML

Type hoàn toàn vô hình với QML. Dùng cho base class ẩn hoặc attached object type. Engine vẫn biết nó tồn tại để coerce các subtype khi cần.

```
cpp
```

```
// Attached object type – QML không cần biết tên này
class MessageBoardAttachedType : public QObject {
    Q_OBJECT
    QML_ANONYMOUS // vô hình với QML
    Q_PROPERTY(bool expired READ expired WRITE setExpired NOTIFY expiredChanged
public:
    explicit MessageBoardAttachedType(QObject *parent = nullptr);
    bool expired() const;
    void setExpired(bool expired);
signals:
    void expiredChanged();
    void published();
};

// Attaching type – cái này mới visible với QML
class MessageBoard : public QObject {
    Q_OBJECT
    QML_ELEMENT
    QML_ATTACHED(MessageBoardAttachedType) // khai báo attached type
public:
    static MessageBoardAttachedType *qmlAttachedProperties(QObject *object) {
        return new MessageBoardAttachedType(object);
    }
};
```

Dùng trong QML:

```
qml
import myapp

Message {
    // Dùng qua qualifier "MessageBoard.", không dùng trực tiếp tên attached type
    MessageBoard.expired: true
    MessageBoard.onPublished: console.log("published!")
}
```

6. `QML_FOREIGN(ExistingClass)` — Register type từ thư viện ngoài

Dùng khi không thể sửa class gốc (3rd party library). Tạo một struct wrapper chứa toàn bộ

macro, struct này không cần instance.

```
cpp

// Giả sử FastDDS có class này và không sửa được:
// class DomainParticipant { ... };

// Tạo file wrapper riêng:
#include <fastdds/dds/domain/DomainParticipant.hpp>
#include <QtQml/qqmlregistration.h>

struct DomainParticipantForeign {
    Q_GADGET
    QML_FOREIGN(eprosima::fastdds::dds::DomainParticipant)
    QML_NAMED_ELEMENT(DDSParticipant)
    QML_UNCREATABLE("Managed by C++ layer")
};
```

CMake:

```
cmake

qt_add_qml_module(myapp
    URI myapp
    VERSION 1.0
    SOURCES domainparticipantforeign.h # chỉ cần .h, không cần .cpp
)
```

Dùng trong QML:

```
qml

import myapp

// Dùng được như type thông thường:
// property DDSParticipant participant: ...
```

7. `QML_VALUE_TYPE(name)` — Value type (`Q_GADGET`, không phải `QObject`)

Dùng cho struct/class **không** kế thừa `QObject`, truyền by-value. Tên trong QML **bắt buộc lowercase**. Không thể là singleton.

```
cpp
```



```
// point.h
#include <QtQml/qqmlregistration.h>

class Point {
    Q_GADGET
    Q_PROPERTY(int x READ x WRITE setX)
    Q_PROPERTY(int y READ y WRITE setY)
    QML_VALUE_TYPE(point) // lowercase bắt buộc
public:
    Point() = default; // phải default-constructible
    Point(const Point &) = default; // phải copy-constructible
    int x() const { return m_x; }
    void setX(int x) { m_x = x; }
    int y() const { return m_y; }
    void setY(int y) { m_y = y; }
private:
    int m_x = 0;
    int m_y = 0;
};
```

Dùng trong QML:

```
qml

import myapp

Rectangle {
    // "point" (lowercase) là value type, dùng như built-in:
    property point center: Qt.point(50, 50)
}
```

Bảng so sánh nhanh

Macro combo	Tạo được từ QML	Visible với QML	Cần import	Dùng khi
<code>QML_ELEMENT</code>	✓	✓	✓	Type thông thường, instantiable
<code>QML_NAMED_ELEMENT(n)</code>	✓	✓	✓	Muốn đổi tên trong QML

Macro combo	Tạo được từ QML	Visible với QML	Cần import	Dùng khi
<code>QML_ELEMENT</code> + <code>QML_SINGLETON</code>	✗ engine tạo	✓	✓	Singleton — dùng thẳng tên class
<code>QML_ELEMENT</code> + <code>QML_UNCREATABLE</code>	✗	✓	✓	Chỉ expose enum / làm kiểu property
<code>QML_ANONYMOUS</code>	✗	✗	N/A	Base class ẩn, attached object type
<code>QML_FOREIGN(T)</code>	tùy	tùy	✓	Wrap 3rd party type
<code>QML_VALUE_TYPE(n)</code>	by-value	✓	✓	Q_GADGET struct, truyền by-value

Lưu ý quan trọng

Về import: Tất cả các type (trừ `QML_ANONYMOUS`) đều **cần được import** trong file QML bằng URI khai báo trong `qt_add_qml_module()`:

```
qml

import calculator          // URI = "calculator" trong CMake
import myapp 1.0           // có thể kèm version
```

`QML_SINGLETON` vs `pragma Singleton`:

	<code>QML_SINGLETON</code> (C++)	<code>pragma Singleton</code> (QML)
Instance do	QQmlEngine tạo	QQmlEngine tạo
Khai báo ở	C++ header	File <code>.qml</code> + CMake property
Constructor	<code>(QObject *parent = nullptr)</code>	N/A
Dùng trong QML	<code>ClassName.property</code>	<code>ClassName.property</code>
CMake extra	Không cần	<code>QT_QML_SINGLETON_TYPE TRUE</code>

Thứ tự kế thừa khi kết hợp QObject và interface khác:

cpp

```
// QObject PHẢI đứng đầu tiên trong danh sách kế thừa
class TaskStore : public QObject, public Store {
    Q_OBJECT    // Q_OBJECT chỉ đặt ở derived class cuối cùng
    QML_ELEMENT
    // ...
};
```

Q_ENUM phải nằm trong class có **Q_OBJECT** hoặc **Q_GADGET**:

cpp

```
// ĐÚNG:
class FilterType : public QObject {
    Q_OBJECT
    QML_ELEMENT
    QML_UNCREATABLE("...")
public:
    enum Type { All, Active, Completed };
    Q_ENUM(Type)    // ← Q_ENUM sau enum declaration
};

// SAI – enum trong namespace thường không expose được:
// namespace FilterType { enum Type { All }; }
```